Mirage19 Data Exploration

In [ ]:
```
import pickle
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import confusion_matrix, classification_report, accu
from collections import Counter
import time
import copy
import multiprocessing
import os

# Configuration
RANDOM_SEED = 2025
```

Load Data

Loading the preprocessed pickle file containing biflows and labels.

In [5]:
```
DATA_PATH = "../dataset/mirage/2019/mirage2019_LOPEZ_lopez_lopez_100P_4F_

print("Loading dataset...")
with open(DATA_PATH, "rb") as f:
    X_raw = np.array(pickle.load(f), dtype=np.float32) # Convert to numpy
    y_raw = np.array(pickle.load(f)) # Convert to numpy array

print(f"Data shape: {X_raw.shape}")
print(f"Labels shape: {len(y_raw)}")

# Example sample
example_sample = 7000
print("\nExample Sample (First 5 packets):\n", X_raw[example_sample][:5])
print("Example Label:", y_raw[example_sample])
```

```
Loading dataset...
Data shape: (96999, 100, 4)
Labels shape: 96999

Example Sample (First 5 packets):
 [[0.0000e+00 2.0300e+02 3.4300e+02 0.0000e+00]
 [1.0000e+00 1.4480e+03 2.3500e+02 2.9068e+04]
 [1.0000e+00 1.4480e+03 2.3500e+02 5.0000e+00]
 [1.0000e+00 1.2000e+03 2.3500e+02 4.0000e+00]
 [1.0000e+00 1.4480e+03 2.3500e+02 1.1000e+01]]
Example Label: AccuWeather
```

Data exploration

Checking the mean number of packets per flow. If dst is -1, it is a padding packet.

In [6]:
```
def count_packets(flow):
    """Calcola il numero di pacchetti validi in un singolo flusso."""
    # Usa la logica ottimizzata: trova l'indice del primo -1, altrimenti
    return next((i for i, packet in enumerate(flow) if packet[0] == -1),

# Esempio
print("Example sample packet len: ", count_packets(X_raw[example_sample])
```

```
Example sample packet len:  45
```

In [7]:
```
X_n_packets = []

X_dir_trimmed = []
X_payloadLen_trimmed = []
X_tcpWin_trimmed = []
X_iat_trimmed = []

for i, flow in enumerate(X_raw):
    num_packets = count_packets(flow)
    X_n_packets.append(num_packets)

    X_dir_trimmed.append(flow[:num_packets, 0])
    X_payloadLen_trimmed.append(flow[:num_packets, 1].sum()) # Questi ult
    X_tcpWin_trimmed.append(flow[:num_packets, 2].mean())
    X_iat_trimmed.append(flow[:num_packets, 3].mean())

X_n_packets = np.array(X_n_packets)

print("Example sample packet length:", X_n_packets[example_sample])

print("Example sample direction:", X_dir_trimmed[example_sample])
print("Example sample payload length:", X_payloadLen_trimmed[example_samp
print("Example sample TCP window:", X_tcpWin_trimmed[example_sample])
print("Example sample IAT:", X_iat_trimmed[example_sample])
```

```
Example sample packet length: 45
Example sample direction: [0. 1. 1. 1. 1. 1. 1. 0. 1. 0. 1. 1. 1. 1. 1. 1.
1. 1. 1. 1. 1. 1. 1. 1.
 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1. 1.]
Example sample payload length: 58570.0
Example sample TCP window: 251.44444
Example sample IAT: 10866.0
```

In [8]:
```
print("Mean packets per flow: ", np.mean(X_n_packets))
print("Standard deviation packets per flow: ", np.std(X_n_packets))
```

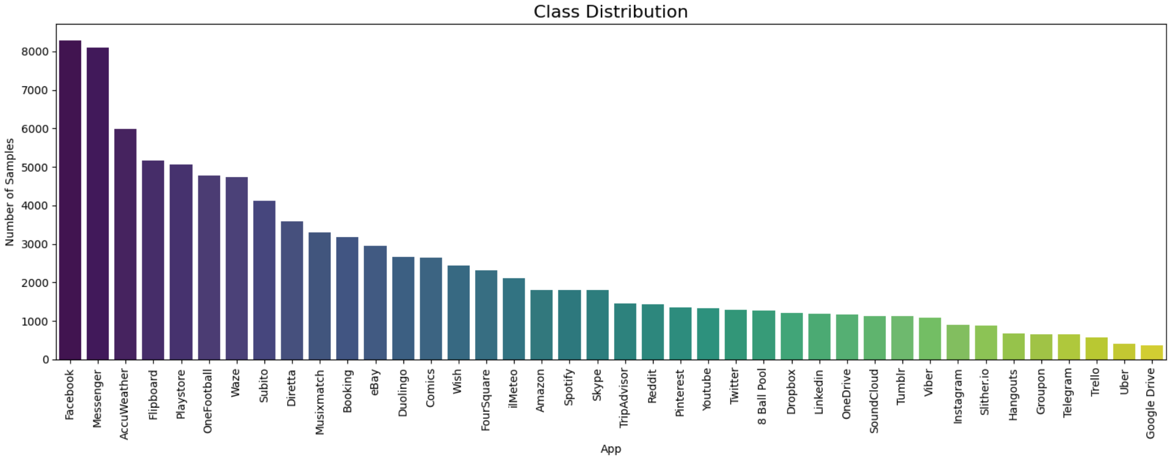
```
Mean packets per flow:  31.128826070371858
Standard deviation packets per flow:  33.90860730419403
```

Analyzing class distribution and feature statistics.

In [13]:
```
# Class Distribution
counts = Counter(y_raw)
df_counts = pd.DataFrame.from_dict(counts, orient='index', columns=['coun

plt.figure(figsize=(15, 6))
sns.barplot(x=df_counts.index, y=df_counts['count'], palette='viridis')
plt.xticks(rotation=90)
```

```python
plt.xlabel("App")
plt.ylabel("Number of Samples")
plt.title("Class Distribution", fontsize=16)
plt.tight_layout()
plt.show()
```

```
/var/folders/tk/347n9cq14gs8tql211z4pvr00000gn/T/ipykernel_68064/62098783.
py:6: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be remove
d in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for
the same effect.

  sns.barplot(x=df_counts.index, y=df_counts['count'], palette='viridis')
```



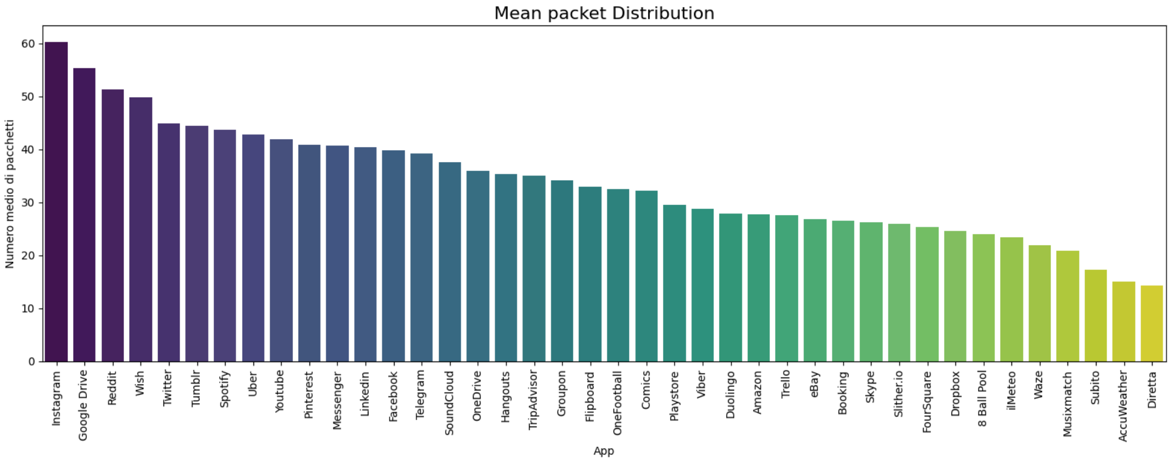
Checking, for each class, the mean number of packets per flow.

In [16]:

```python
# Mean packet pre flow
data = {
    'y': y_raw,
    'x': X_n_packets,
    # 'dir': X_dir,
    'payloadLen': X_payloadLen_trimmed,
    'tcpWin': X_tcpWin_trimmed,
    'iat': X_iat_trimmed,
}
df = pd.DataFrame(data)

df_media = df.groupby('y')['x'].mean().reset_index()
df_media.columns = ['Etichetta', 'Media_X']
df_media = df_media.sort_values('Media_X', ascending=False)

plt.figure(figsize=(15, 6))

sns.barplot(
    x='Etichetta',
    y='Media_X',
    data=df_media,
    palette='viridis'
)

plt.xticks(rotation=90)
plt.title("Mean packet Distribution", fontsize=16)
plt.xlabel("App")
plt.ylabel("Numero medio di pacchetti")
```

```
plt.tight_layout()
plt.show()
```

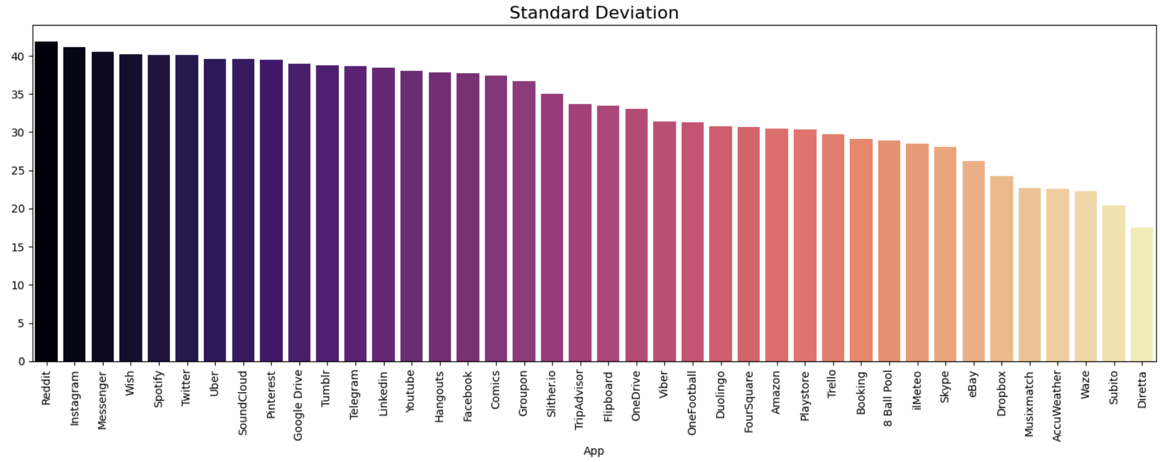
```
/var/folders/tk/347n9cq14gs8tql211z4pvr00000gn/T/ipykernel_68064/419174386
5.py:18: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be remove
d in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for
the same effect.

  sns.barplot(
```



Standard Deviation for each class

In [22]:
```python
df_std = df.groupby('y')['x'].std().reset_index()

# Rinomina le colonne
df_std.columns = ['Etichetta', 'Deviazione_Standard_X']

# Ordina in base alla Deviazione Standard in modo decrescente
df_std = df_std.sort_values('Deviazione_Standard_X', ascending=False)


# --- 3. Visualizzazione con Seaborn ---
plt.figure(figsize=(15, 6))

# Usa sns.barplot con il DataFrame df_std ordinato
sns.barplot(
    x='Etichetta',
    y='Deviazione_Standard_X',
    data=df_std,
    palette='magma'
)

# Impostazioni finali del grafico
plt.xticks(rotation=90)
plt.title("Standard Deviation", fontsize=16)
plt.xlabel("App")
plt.ylabel(" ")
plt.tight_layout()
plt.show()
```

```
/var/folders/tk/347n9cq14gs8tql211z4pvr00000gn/T/ipykernel_68064/38915223
4.py:14: FutureWarning:

Passing `palette` without assigning `hue` is deprecated and will be remove
d in v0.14.0. Assign the `x` variable to `hue` and set `legend=False` for
the same effect.

  sns.barplot(
```



In [23]:
```
df_analisi_completa = df.groupby('y')['x'].agg(
    [
        'mean',       # Media
        'std',        # Deviazione Standard
        'median',     # Mediana
        'min',        # Minimo
        'max',        # Massimo
        pd.Series.skew, # Asimmetria (Skewness)
        pd.Series.kurt  # Curtosi (Kurtosis)
    ]
).reset_index()

print(df_analisi_completa)
```

	y	mean	std	median	min	max	skew	kurt
0	8 Ball Pool	23.980453	28.946398	11.0	4	100	2.012787	2.407426
1	AccuWeather	14.990144	22.571888	8.0	1	100	3.014657	8.186634
2	Amazon	27.711209	30.447291	14.0	3	100	1.616594	1.070768
3	Booking	26.505660	29.098713	13.0	3	100	1.802087	1.721809
4	Comics	32.148429	37.425825	11.0	2	100	1.125346	−0.560759
5	Diretta	14.330168	17.477465	11.0	1	100	3.145108	10.952170
6	Dropbox	24.547855	24.294373	14.0	6	100	2.142768	3.601152
7	Duolingo	27.858753	30.826574	13.0	3	100	1.559101	0.907863
8	Facebook	39.857384	37.731260	20.0	1	100	0.735624	−1.175274
9	Flipboard	32.962319	33.460705	17.0	1	100	1.122284	−0.282696
10	FourSquare	25.347355	30.665028	11.0	1	100	1.750522	1.463315
11	Google Drive	55.318560	38.976644	48.0	2	100	0.080037	−1.773343
12	Groupon	34.194145	36.644512	13.0	2	100	1.049828	−0.685433
13	Hangouts	35.392387	37.788671	14.0	1	100	0.914302	−0.939978
14	Instagram	60.335177	41.167317	74.5	2	100	−0.204556	−1.817216
15	Linkedin	40.345178	38.415584	19.0	1	100	0.715609	−1.255655
16	Messenger	40.778614	40.532236	20.0	1	100	0.568117	−1.432463
17	Musixmatch	20.896845	22.728128	12.0	2	100	2.479760	5.419275
18	OneDrive	35.869528	33.036507	18.0	1	100	1.089463	−0.430317
19	OneFootball	32.456526	31.328801	17.0	2	100	1.266235	0.124493
20	Pinterest	40.841988	39.434659	17.0	1	100	0.694987	−1.346110
21	Playstore	29.555424	30.342764	16.0	1	100	1.493341	0.839358
22	Reddit	51.286806	41.917376	28.5	2	100	0.198655	−1.853038
23	Skype	26.205327	28.127259	14.0	1	100	1.739948	1.742563
24	Slither.io	25.998871	35.044685	6.0	2	100	1.422837	0.279175
25	SoundCloud	37.577265	39.561966	13.0	2	100	0.827998	−1.180571
26	Spotify	43.775567	40.150387	22.0	2	100	0.464251	−1.567810
27	Subito	17.236951	20.442606	11.0	2	100	3.037956	8.675438
28	Telegram	39.198450	38.675040	19.0	1	100	0.691972	−1.239752

```
29       Trello  27.563478  29.752212    13.0    2  100  1.732102   1.444
624
30  TripAdvisor  35.074227  33.681737    18.0    1  100  1.059782  -0.437
005
31       Tumblr  44.507583  38.754311    23.0    2  100  0.577697  -1.477
903
32      Twitter  44.841821  40.094854    18.0    2  100  0.542388  -1.586
859
33         Uber  42.811275  39.613441    18.0    2  100  0.636177  -1.468
735
34        Viber  28.762037  31.402907    14.0    2  100  1.494920   0.707
050
35         Waze  21.914226  22.309263    12.0    1  100  2.247662   4.663
162
36         Wish  49.792778  40.202221    35.0    1  100  0.251741  -1.717
160
37      Youtube  41.916479  38.026878    24.0    3  100  0.676233  -1.298
276
38         eBay  26.878634  26.219060    15.0    1  100  1.746603   1.969
475
39      ilMeteo  23.474657  28.502651    11.0    1  100  1.833249   2.115
751
```

Preprocessing

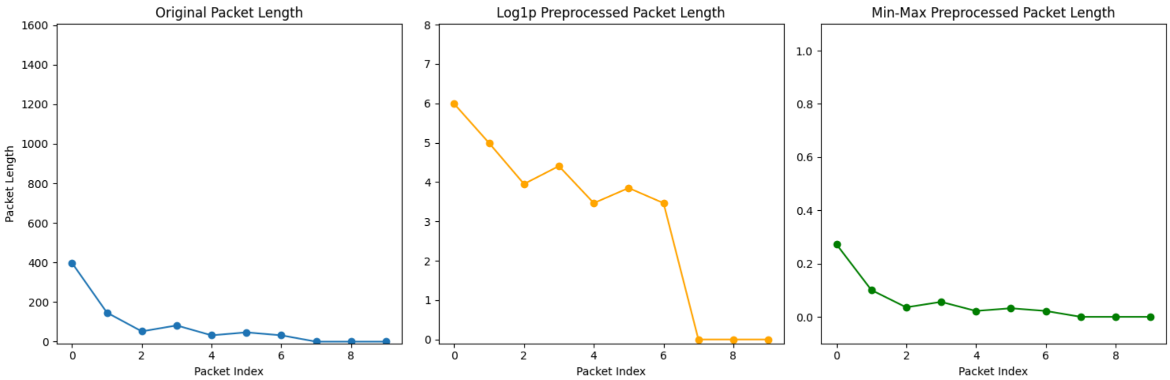
In [24]:

```python
from sklearn.preprocessing import MinMaxScaler

N_PACKETS = 10
N_FEATURES = 4

def log1pPreproc(X):
    """
    Handles padding and normalizes features.
    X shape: (N, 36, 4)
    Features: [DIR, PL, TCPWIN, IAT]
    """
    X_proc = X.copy().astype(np.float32)

    # Mask for padding (using PL at index 1)
    is_padding = (X_proc[:, :, 1] == -1)

    # Set all features to 0 where is_padding is True
    mask = np.repeat(is_padding[:, :, np.newaxis], 4, axis=2)
    X_proc[mask] = 0

    # Apply Log1p to PL (1), TCPWIN (2), IAT (3)
    X_proc[:, :, 1:] = np.log1p(np.maximum(X_proc[:, :, 1:], 0))

    return X_proc[:, :N_PACKETS, :]


def minmaxPreproc(X):
    X_proc = X.copy().astype(np.float32)
    # min max scaler
    scaler = MinMaxScaler(feature_range=(0, 1))
    scaler.fit(np.reshape(X_proc, [-1, N_FEATURES]))
    X_proc = scaler.transform(np.reshape(X_proc, [-1, N_FEATURES]))
    X_proc = np.reshape(X_proc, [-1, X.shape[1], N_FEATURES])
    return X_proc[:, :N_PACKETS, :]
```

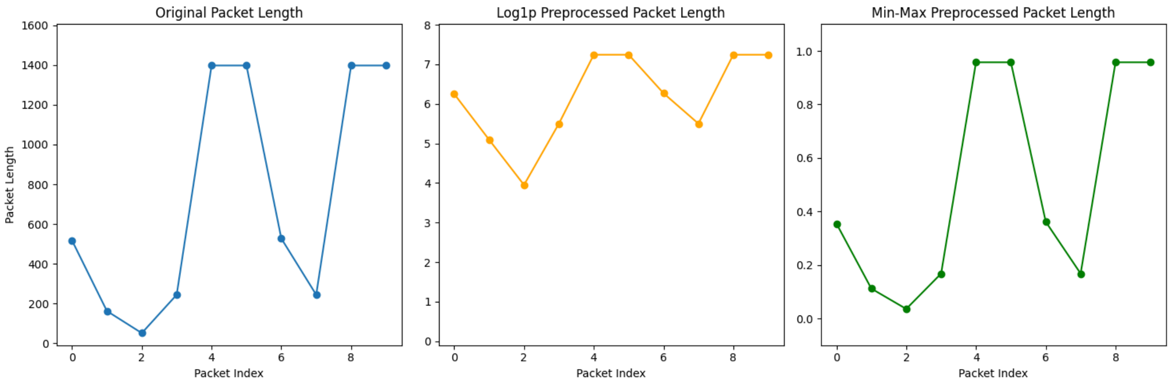
```
X_log = log1pPreproc(X_raw)
X_minmax = minmaxPreproc(X_raw)

# for i in range(20):
# numero random da 0 a len(X_raw)-1
random_sample = np.random.randint(0, len(X_raw))

for random_sample in [52180, 44261]:
    print("Random sample index:", random_sample, "App label:", y_raw[rand

    # pl list
    pl = X_raw[random_sample, :, 1]
    max = np.max(pl[pl != -1])
    min = np.min(pl[pl != -1])
    print("Original PL - min:", min, "max:", max, "delta:", max - min)

    # log1p pl
    pl_log = X_log[random_sample, :, 1]
    max_log = np.max(pl_log)
    min_log = np.min(pl_log[pl_log != 0])
    print("Log1p PL - min:", min_log, "max:", max_log, "delta:", max_log

    # minmax pl
    pl_minmax = X_minmax[random_sample, :, 1]
    max_minmax = np.max(pl_minmax)
    min_minmax = np.min(pl_minmax[pl_minmax != 0])
    print("MinMax PL - min:", min_minmax, "max:", max_minmax, "delta:", m

    # Plot Packet Length of random sample before and after preprocessing
    plt.figure(figsize=(15, 5))
    plt.subplot(1, 3, 1)
    plt.plot(X_raw[random_sample, :N_PACKETS, 1], marker='o')
    plt.title("Original Packet Length")
    plt.xlabel("Packet Index")
    plt.ylabel("Packet Length")
    plt.ylim(-10, np.max(X_raw[:, :, 1]) * 1.1)
    plt.subplot(1, 3, 2)
    plt.plot(X_log[random_sample, :, 1], marker='o', color='orange')
    plt.title("Log1p Preprocessed Packet Length")
    plt.xlabel("Packet Index")
    plt.ylim(-0.1, np.max(X_log[:, :, 1]) * 1.1)
    plt.subplot(1, 3, 3)
    plt.plot(X_minmax[random_sample, :, 1], marker='o', color='green')
    plt.title("Min-Max Preprocessed Packet Length")
    plt.xlabel("Packet Index")
    plt.ylim(-0.1, 1.1)
    plt.tight_layout()
    plt.show()

# 52180 44261

# stessa cosa ma su dir feature
for random_sample in [52180, 44261]:
    print("Random sample index:", random_sample, "App label:", y_raw[rand
    # Plot Direction of random sample before and after preprocessing
    plt.figure(figsize=(15, 5))
    plt.subplot(1, 3, 1)
    plt.plot(X_raw[random_sample, :N_PACKETS, 0], marker='o')
```

```
    plt.title("Original Direction")
    plt.xlabel("Packet Index")
    plt.ylabel("Direction")
    plt.ylim(-1.1, 1.1)
    plt.subplot(1, 3, 2)
    plt.plot(X_log[random_sample, :, 0], marker='o', color='orange')
    plt.title("Log1p Preprocessed Direction")
    plt.xlabel("Packet Index")
    plt.ylim(-1.1, 1.1)
    plt.subplot(1, 3, 3)
    plt.plot(X_minmax[random_sample, :, 0], marker='o', color='green')
    plt.title("Min-Max Preprocessed Direction")
    plt.xlabel("Packet Index")
    plt.ylim(-1.1, 1.1)
    plt.tight_layout()
    plt.show()
```

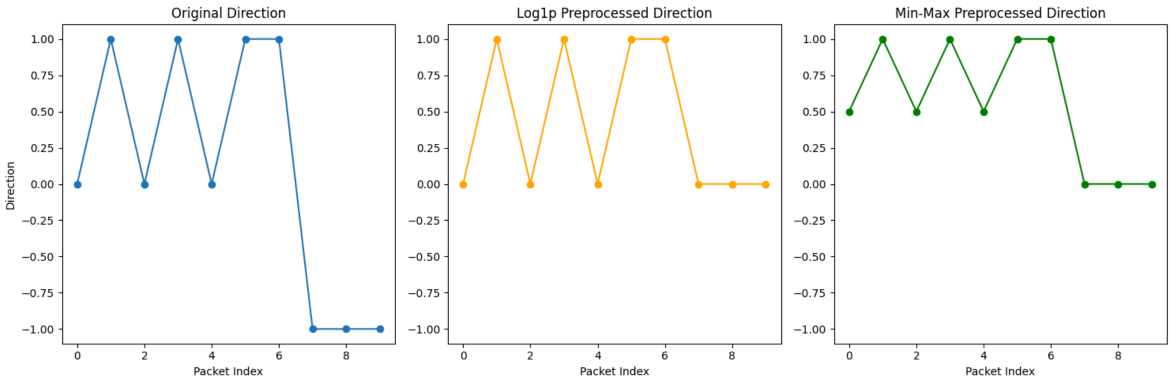
```
Random sample index: 52180 App label: Messenger
Original PL - min: 31.0 max: 397.0 delta: 366.0
Log1p PL - min: 3.465736 max: 5.986452 delta: 2.5207162
MinMax PL - min: 0.021902807 max: 0.27241617 delta: 0.25051337
```




```
Random sample index: 44261 App label: ilMeteo
Original PL - min: 51.0 max: 1460.0 delta: 1409.0
Log1p PL - min: 3.9512436 max: 7.243513 delta: 3.2922695
MinMax PL - min: 0.03559206 max: 0.95756334 delta: 0.92197126
```




```
Random sample index: 52180 App label: Messenger
```



Random sample index: 44261 App label: ilMeteo